

(index.html)

Componentes visuales (Interfaces
mediante documentos XML)

Fuente estándar



A+

A-



Leer (es)

Flotar

Info (<https://batexego.bilateria.org>)

DI07 - Componentes visuales (interfaces mediante documentos XML).



Creación de componentes visuales: Introducción.



Caso práctico

Como siempre, en BK Programación están que no paran, **Juan y María** andan liados programando una aplicación para una empresa de venta por Internet.

-Claro, pero es que necesitamos que la interfaz disponga de un temporizador que podamos programar para implementar la cuenta atrás del tiempo que le queda a la oferta.

-Es cierto, pero, es un trabajo sencillo, no nos debe costar demasiado hacer eso...

-Cierto, además creo que hicimos algo parecido en el proyecto para el programa de juegos educativos para el instituto, necesitábamos regular el tiempo que se le daba a los alumnos para resolver el juego. Estaría muy bien recuperar ese código.

-Lo que estaría verdaderamente bien sería poder usar ese código directamente, sin necesidad de tener que buscarlo y adaptarlo...

En realidad, lo que **Juan y María** necesitan es adaptar el código del temporizador, que ya han programado, de manera que puedan usarlo desde la misma herramienta con la que programan sus aplicaciones, y para ello una de las mejores opciones es el uso de **componentes**.



Q Ministerio de Educación y
Formación Profesional
(Elaboración propia)



Q Ministerio de Educación y
Formación Profesional.
(Elaboración propia)



Materiales formativos de FP Online propiedad del Ministerio de Educación y Formación Profesional.

 [Aviso Legal](#)

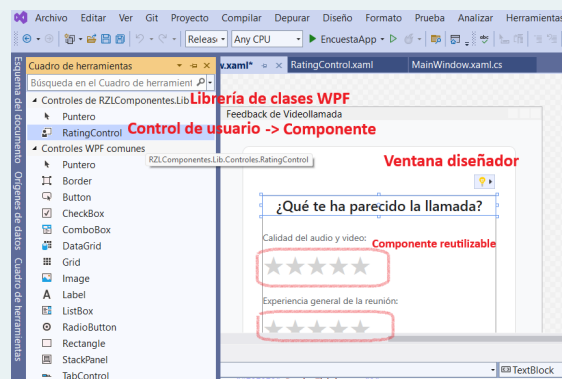
Concepto de componente. Características.



Caso práctico

Ada ha escuchado la conversación de sus compañeros, e interviene para explicarles qué es un componente:

-Un componente es una pieza de hardware o software pequeña, que tiene un comportamiento específico y dispone de una interfaz que permite que se inserte fácilmente. Por ejemplo, una etiqueta o un botón que se basan en las etiquetas o botones estándares de la barra de herramientas de un IDE adaptados a las necesidades específicas de una aplicación y que pueden ser reutilizada es un componente. Tienen la ventaja de que pueden especializarse mucho y al estar probadas de antemano sabemos que su funcionamiento es correcto por lo que facilitan la programación de aplicaciones nuevas.



(componentesyControles.png)

Q IDE Visual Studio: proyecto WPF (

CC BY-SA)

Un **componente software** es una clase creada para ser reutilizada y que puede ser manipulada por una herramienta de desarrollo de aplicaciones visual. Se define por su **estado** que se almacena en un conjunto de **propiedades**, las cuales pueden ser modificadas para adaptar el componente al programa en el que se inserte. También tiene un **comportamiento** que se define por los eventos ante los que responde y los **métodos** que ejecuta ante dichos eventos.

Los eventos son las acciones que los usuarios van a provocar sobre los componentes.

Un subconjunto de los atributos y los métodos forman la **interfaz** del componente.

Para que pueda ser distribuida, se empaqueta con todo lo necesario para su correcto funcionamiento, quedando independiente de otras bibliotecas o componentes.



Citas para pensar

Un componente es una unidad de composición de software, que posee un conjunto de interfaces y un conjunto de requisitos, y que ha de poder ser desarrollado, adquirido, incorporado al sistema y compuesto de otros componentes, de forma independiente en tiempo y en espacio.

Szypersky, 1998

Para que una clase sea considerada un componente debe cumplir ciertas normas:

- Debe poder modificarse para adaptarse a la aplicación en la que se integra.
- Debe tener persistencia, es decir, debe poder guardar el estado de sus propiedades cuando han sido modificadas.
- Debe tener introspección, es decir, debe permitir a un IDE que pueda reconocer ciertos elementos de diseño como los nombres de las funciones miembros o métodos y definiciones de las clases, y devolver esa información.
- Debe poder gestionar **eventos**.

El desarrollo basado en componentes tiene, además, las siguientes **ventajas**:

- Es mucho más sencillo, se realiza en menos tiempo y con un coste inferior.
- Se disminuyen los errores en el software ya que los componentes se deben someter a un riguroso control de calidad antes de ser utilizados.



Reflexiona

Ya has creado algunas interfaces gráficas. El procedimiento consiste en seleccionar los controles gráficos de la interfaz y posicionarlos en un lienzo que será la interfaz. Todos estos controles **cumplen con los requisitos para ser componentes, de hecho lo son**, son elementos reutilizables que pueden ser manejados por una herramienta de desarrollo visual. Por ejemplo, se puede modificar su tamaño o color para adaptarlo a la interfaz y estos cambios permanecen después de cerrarla, además tienen una interfaz formada por un conjunto de métodos y propiedades accesibles desde la paleta de propiedades y la capacidad de responder a eventos, por ejemplo, el clic del ratón.



Autoevaluación

Imagina una herramienta de desarrollo, están compuestas de un conjunto de ventanas que podemos mostrar u ocultar, cambiar su tamaño, etc., de modo que al cerrar la aplicación y volverla a abrir se mantiene la estructura que tenía al cerrarse. ¿Con qué característica de los componentes relacionas esto?

- ☐ Con la persistencia.
- ☐ Con la introspección.
- ☐ Con la gestión de eventos.

Así es, la persistencia, que permite que los cambios en las propiedades de las ventanas se mantengan de una sesión a otra.

Falso, la introspección se relaciona con la capacidad de una herramienta de desarrollo de reconocer los elementos de la interfaz de un componente.

Falso, la gestión de eventos se relaciona con la capacidad de reacción del componente ante ciertos estímulos.

Solución

1. Opción correcta (#answer-188_63)
2. Incorrecto (#answer-188_79)
3. Incorrecto (#answer-188_82)

Elementos de un componente: propiedades y atributos.



Caso práctico

Ada continúa explicando a sus compañeros las características más básicas de un componente:

-Un componente software no deja de ser una clase que vamos a tratar de una forma un poco diferente, así que como todas las clases se compone de un conjunto de elementos de tipos básicos a los que se llama atributos, pero si los atributos cumplen ciertas características pasarán a ser propiedades. Por ejemplo, una etiqueta es un componente las características que la definen son sus propiedades como el color o el tipo de la fuente. Además, tiene un conjunto de métodos o funciones que permiten modificar las propiedades, y para implementar la relación con otros elementos software o con el usuario le podemos definir una serie de eventos.



Q Ministerio de
Educación y Formación
Profesional (Elaboración
propia)

Como en cualquier clase, un componente tendrá definido un estado a partir de un conjunto de **atributos**. Los atributos son variables definidas por su nombre y su tipo de datos que toman valores concretos. Normalmente los atributos son privados y no se ven desde fuera de la clase que implementa el componente, se usan sólo a nivel de programación.

Las propiedades son un tipo específico de atributos que representan características de un componente que afectan a su apariencia o a su comportamiento. Son accesibles desde fuera de la clase y forman parte de su interfaz. Suelen estar asociadas a un atributo interno.

Las propiedades de un componente pueden examinarse y modificarse mediante métodos o funciones miembro, que acceden a dicha propiedad, y pueden ser de dos tipos:



Q Ministerio de Educación y
Formación Profesional (Elaboración
propia)

- **getter:** permiten leer el valor de la propiedad. Tienen la estructura:

```
public <TipoPropiedad> get<NombrePropiedad>( )
```

Si la propiedad es booleana el método getter se implementa así:

```
public boolean is<NombrePropiedad>()
```

- **setter:** permiten establecer el valor de la propiedad. Tiene la estructura:

```
public void set<NombrePropiedad>(<TipoPropiedad> valor)
```

Si una propiedad no incluye el método set entonces es una propiedad de **sólo lectura**.

Por ejemplo, si estamos generando un componente visual en C# que sirva para permitir a un usuario hacer una valoración en un cuestionario, tendremos, entre otras, la propiedad del Valor que el usuario selecciona a través de la interfaz del componente con los siguientes métodos asociados:

```
namespace Componentes.Lib.Controles
{
    ///
    /// Lógica de interacción para RatingControl.xaml
    ///
    public partial class RatingControl : UserControl
    {
        //CREAMOS LA PROPIEDAD "Value"
```



```
public static readonly DependencyProperty ValueProperty =
    DependencyProperty.Register(
        "Value",
        typeof(int),
        typeof(RatingControl),
        new PropertyMetadata(0, OnValueChanged));

public int Value
{
    get { return (int)GetValue(ValueProperty); }
    set { SetValue(ValueProperty, value); }
}

public RatingControl()
{
    InitializeComponent();
}
}
```

Componentes nativos del framework WPF .NET.

En el ecosistema nativo de WPF en Visual Studio, tenemos tres caminos principales para desarrollar componentes, dependiendo de cuánto quieras reutilizar y personalizar:

1. User Controls (Controles de Usuario): Es la opción más común y sencilla. Combinas (composición) controles existentes (como un Button y un TextBox) en un nuevo grupo con su propio archivo .xaml y .xaml.cs. Es ideal para bloques de interfaz que se repiten en la misma aplicación.
2. Custom Controls (Controles Personalizados): Se crean extendiendo una clase existente (como Control o ItemsControl). A diferencia del UserControl, el diseño se define en un archivo genérico (Generic.xaml). Son más complejos de crear pero son los mejores si buscas componentes totalmente reutilizables en diferentes proyectos, ya que permiten aplicar plantillas (ControlTemplate) externas.
3. Adorners (Adornadores): Son componentes visuales especiales que "flotan" sobre otros elementos para añadir decoración o funcionalidad (como los "tiradores" de redimensión de una imagen, etc.).



Para gestionar la lógica de estos componentes de forma limpia, lo estándar es seguir el patrón MVVM (Model-View-ViewModel), utilizando el Binding de datos y los Commands para que la interfaz y el código estén desacoplados.

¿Por cuál decidimos? ¿UserControl y/o Custom Control?

- Si vas a combinar varios controles existentes (un TextBox + un Botón) de forma rápida, usa un UserControl.
- Si quieres un control "atómico" (un botón con un diseño único, un selector de fecha especial) que sea altamente personalizable, quédate con el Custom Control.

Característica	User Control (.xaml + .cs)	Custom Control (.cs + Generic.xaml)
Composición	Combinas controles existentes (ej: Label + TextBox).	Creas un control desde la base o extiendes uno existente.
Diseño	El diseño está "fijo" dentro del control.	El diseño es una "plantilla" (Template) intercambiable.
Facilidad	Muy fácil de diseñar visualmente (Drag & Drop).	Curva de aprendizaje más alta (usas TemplateBinding).
Reutilización	Ideal para piezas específicas de una App.	Ideal para librerías de UI que se usarán en muchas Apps distintas.

1.- Lenguajes basados en XML.

¿Qué es XML?

XML (eXtensible Markup Language o Lenguaje de Marcado eXtensible) es un lenguaje basado en texto de definición de documentos, que permite representar información estructurada de gran variedad de documentos.

Al estar puramente basado en texto, un documento **XML** será un archivo plano en el que, a través de una serie de elementos propios se definen aspectos del documento a generar, pero no de su diseño propiamente dicho, sino de su estructura.



Q Ministerio de Educación y Formación Profesional. (Elaboración propia)

Orígenes

Hacer independiente la estructura de un documento de la constante evolución de las herramientas que se usan para visualizarlo ha sido, desde hace tiempo, foco de interés de los desarrolladores y desarrolladoras de aplicaciones informáticas. En los años sesenta, Charles F. Goldfab, por encargo de IBM desarrolló el lenguaje **GML**, Generalized Markup Language, cuyo objetivo era describir la estructura de los documentos de forma que el resultado no dependiese de una determinada plataforma o una aplicación específica. De la evolución de **GML** surgió **SGML** que se convirtió en el estándar internacional **ISO 8879**. Sin embargo, tuvo difusión tan sólo en medios académicos y de la administración, pero no tuvo demasiado éxito entre los usuarios medios debido a su dificultad.

El hito que supuso la amplia difusión de las tecnologías de la comunicación y la circulación de documentos electrónicos entre cualquier punto del mundo fue la creación de la World Wide Web (Tim Berners-Lee, en 1990), y, asociado a ello la aparición de **HTML**, lenguaje de descripción de documentos basado en **SGML** para la web.

HTML es un lenguaje que permite combinar en un solo documento el contenido con el formato, por ejemplo:

- Si en un documento **HTML** se añade el siguiente código: "El padre del lenguaje **GML** fue **Charles F. Goldfab**, que lo desarrolló por encargo de **IBM**."
- Conseguiré algo parecido a esto: El padre del lenguaje **GML** fue **Charles F. Goldfab**, que lo desarrolló por encargo de **IBM**.

Ya que encerrar un texto entre **** y **** equivale a aplicar el formato negrita. Estos elementos de **HTML** como **** y **** se denominan etiquetas, cada etiqueta tiene un significado en el diseño del documento final o en su estructura.

XML sin embargo, **no incluye ninguna orientación al diseño** es puramente un

lenguaje estructural cuyo objetivo es definir cómo se organiza la información en un documento, el diseño se definirá después, de esta forma se hace independiente al documento de la plataforma, e incluso de la aplicación con la que se va a visualizar (Web, impreso, como documento de texto, y más). Por ejemplo, si definimos una página web con XML podremos verla fácilmente en un navegador web, en un teléfono móvil, o, incluso, en un lector Braille, sin modificar el documento de base. Lo que cambia es la forma en que se interpreta el contenido.

XML nace con el objetivo de convertirse en un estándar para el intercambio de información estructurada entre diferentes plataformas: Windows, Android, Linux, iOS, etc. Se puede utilizar en bases de datos, editores de texto, cálculos, dispositivos móviles, etc... Actualmente es la base para la creación de interfaces gráficas de muchos generadores de códigos o IDEs como Visual Studio, Android Studio, NetBeans y Eclipse.



Reflexiona

XML no incluye ninguna orientación al diseño. Esto quiere decir que no vamos a hacer referencia a ningún aspecto relativo a la aplicación final en la que visualicemos el documento. Entonces ¿cómo crees que podemos usar el mismo archivo para ver en un navegador que en un teléfono móvil? ¿Qué necesitaremos para visualizar ese contenido de formas diferentes?

Mostrar retroalimentación

Efectivamente, los archivos XML se pueden visualizar en diferentes plataformas, como navegadores web o teléfonos móviles, para salvar las diferencias se debe acompañar de un archivo auxiliar, que interpreta cómo debe visualizarse el contenido XML en cada dispositivo. Este archivo se denomina XSL y contiene la definición de cómo se verán los elementos XML en un dispositivo concreto.



Autoevaluación

¿Cuál de las siguientes afirmaciones es correcta?

- ☐ SGML ha sido un lenguaje de muy amplia difusión desde su creación en los años ochenta.
- ☐ La estructura de un documento XML depende totalmente de la aplicación con la que se visualizará.

- ☐ El lenguaje HTML es un lenguaje XML puro.
- ☐ Con HTML podemos indicar cuestiones tales como el color de la fuente o si vamos a visualizar un texto centrado.

No es correcta, se consideraba este lenguaje difícil de aprender y de usar por lo que no tuvo mucho auge.

No es correcta, precisamente el principal objetivo de XML es hacer independiente la estructura del documento de la aplicación final donde será visualizado.

No es la respuesta correcta porque con HTML podemos hacer referencia a cuestiones de diseño del documento final, y este aspecto no se contempla en un documento XML.

Es correcta ya que, a pesar de estar basado en XML, HTML si hace referencia a cuestiones relativas al diseño de la página web como por ejemplo el formato de la fuente o el párrafo.

Solución

1. Incorrecto (#answer-6_63)
2. Incorrecto (#answer-6_74)
3. Incorrecto (#answer-6_77)
4. Opción correcta (#answer-6_80)



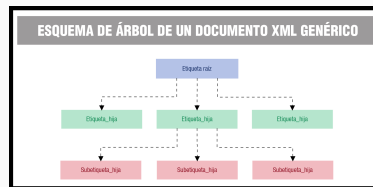
Para saber más

Puedes encontrar un completo artículo sobre XML en este enlace:

[Artículo sobre XML.](#)

1.1.- Etiquetas.

XML es un tanto parecido a HTML en el sentido de que utiliza etiquetas para definir elementos dentro de una estructura, con la diferencia de que cuando usamos HTML se trabaja con una serie de etiquetas predefinidas, y en XML se pueden definir según convenga a las necesidades del proyecto.



Q Ministerio de Educación y Formación
Profesional. (Elaboración propia)

Las etiquetas están delimitadas por ángulos (<,>) e identifican el contenido que delimitan. Pueden tener atributos. Siguen la estructura:

<etiqueta atributo="valor">Contenido</etiqueta>

Las etiquetas en un documento XML tienen siempre **etiqueta de cierre**, de manera que toda la información referente al elemento queda comprendida entre ambas.

Siempre existe una **etiqueta raíz**, que hace referencia al tipo de objeto que se describe. El resto son características o elementos del objeto, y se colocan anidadas dentro de la etiqueta raíz como etiquetas hijas. Por eso decimos que un documento XML tiene **estructura de árbol**.

Las etiquetas en un documento XML tienen siempre etiqueta de cierre, de manera que toda la información referente al elemento queda comprendida entre ambas.

Siempre existe una etiqueta raíz, que hace referencia al tipo de objeto que se describe. El resto son características o elementos del objeto, y se colocan anidadas dentro de la etiqueta raíz como etiquetas hijas. Por eso decimos que un documento XML tiene estructura de árbol.

Estructura de árbol de un documento XML ge

Documento XML

```
<Etiqueta_raiz>
  <etiqueta_hija>
    <subetiqueta_hija>...</subetiqueta_hija>
    <subetiqueta_hija>...</subetiqueta_hija>
    ...
  </etiqueta_hija>
</Etiqueta_raiz>
```

Por **ejemplo**, si queremos usar XML para almacenar la información de nuestra agenda de contactos podríamos definir una etiqueta llamada contacto para cada entrada de la agenda, y dentro de los contactos otras etiquetas para el nombre, teléfono, fecha de nacimiento, grupo, entre otras. Si tengo un amigo que se llama Javier, cuyo número de teléfono es 637-059874 y que nació el 14 de diciembre de 1982, almacenaría la información referente a él en mi agenda de la siguiente manera:

Estructura de árbol del documento XML para

Documento XML

```
<agenda>
<contacto>
  <nombre>Javier</nombre>
  <teléfono>637059874</teléfono>
  <fecha_nacimiento>14-12-1982</fecha_nacimiento>
  <grupo>Amigos</grupo>
</contacto>
</agenda>
```

Donde la etiqueta raíz sería agenda, contacto sería una etiqueta hija y nombre, teléfono, fecha_nacimiento y grupo funcionarían como subetiquetas.

1.2.- Atributos y valores.

Se dice que un elemento XML tiene contenido cuando se ha añadido algún texto entre la etiqueta de apertura y cierre. Sin embargo, pueden darse casos en los que no se pueda almacenar toda la información pertinente del contenido sólo en el texto que encierran las etiquetas.

Cuando necesitamos añadir información adicional a un elemento XML de alguna manera modificamos la etiqueta añadiéndole **atributos**.

<etiqueta atributo="valor">Contenido</etiqueta>

Los atributos permiten proporcionar información adicional sobre el elemento. Por ejemplo, puedo precisar si el teléfono que he guardado para mi amigo Javier es su teléfono fijo, su móvil o el teléfono del trabajo añadiendo el atributo tipo:



Q INTEFP (CC BY-NC-SA)

```
<agenda>
  <contacto>
    <nombre>Javier</nombre>
    <teléfono tipo="móvil">637059874</teléfono>
    <fecha_nacimiento>14-12-1982</fecha_nacimiento>
    <grupo>Amigos</grupo>
  </contacto>
</agenda>
```

No siempre ocurre que cumpliendo los requisitos anteriormente comentados el documento XML sea correcto. Para asegurarnos de que un documento XML esté bien formado debe cumplir las siguientes reglas:

- Debe existir un elemento raíz.

- Todos los elementos XML deben tener su correspondiente etiqueta de cierre.
- Es sensible a mayúsculas.
- El anidamiento debe hacerse conforme a la estructura del árbol del documento. Es decir, si abro la etiqueta y a continuación se cierran en sentido inverso, no puedo cerrar antes y después
- Los valores de los atributos van entrecomillados siempre.

A la hora de poner nombre a las etiquetas se deben seguir las siguientes recomendaciones:

- Se pueden usar letras, número y otros caracteres.
- No se puede empezar por un número o signo de puntuación.
- No se puede empezar por las letras XML .
- Los nombres no pueden contener espacios en blanco.



Autoevaluación

Texto de la pregunta: ¿cuál es la expresión correcta?

- ☐ `<fecha de nacimiento>14-12-1982</fecha de nacimiento>`
- ☐ `<teléfono tipo=movil>637059874</teléfono>`
- ☐

```
<contacto>
<nombre>
Javier
</contacto>
</nombre>
```
- ☐ `<email tipo="personal">javier@gmail.com</email>.`

No es correcta porque los nombres de las etiquetas no pueden contener espacios en blanco.

No es correcta porque los valores de los atributos deben ir entrecomillados siempre.

No es correcta porque no cumple con el anidamiento de etiquetas conforme a la estructura de árbol del documento.

Correcta.

Solución

1. Incorrecto (#answer-14_63)
2. Incorrecto (#answer-14_92)
3. Incorrecto (#answer-14_95)
4. Opción correcta (#answer-14_98)

2.- Lenguajes de descripción de interfaces basados en XML.

El desarrollo de interfaces de usuario es una actividad compleja que requiere de la intervención de múltiples factores. El desarrollador o desarrolladora se enfrenta a una serie de hitos que debe superar:

- En primer lugar, se debe definir lo que se espera de la interfaz, que requisitos debe cubrir y el entorno en el que se integra.
- También se debe tener en cuenta la variedad de herramientas y lenguajes de programación que exigen un elevado nivel de conocimientos.
- Por otra, parte necesitamos interfaces (así como las aplicaciones subyacentes) para múltiples contextos de uso, que dependen del usuario (por su idioma, preferencias o posibles discapacidades), de la plataforma (ordenadores personales, móviles, miniportátiles, pantallas, etc.), del modo de presentación de la interfaz (gráfica o texto) o del dispositivo final (pantallas de ordenador, pizarras digitales, proyectores, etc.).



Q Ministerio de Educación y

Formación Profesional. (Elaboración propia)

2.1.- Proceso de elaboración de las interfaces.

En unidades anteriores, para desarrollar una interfaz hemos empleado el mismo lenguaje usamos para implementar la funcionalidad de la aplicación C#. Hemos visto que tiene como ventaja la sencillez, puesto que no es preciso adquirir nuevos conocimientos. La creación de la interfaz se integra en el proceso de desarrollo de la aplicación y, normalmente, no es necesario trabajar con herramientas diferentes, ni en la programación ni posteriormente a la hora de compilar o ejecutar el programa. Sin embargo, tiene como principal desventaja la dependencia de la plataforma, del dispositivo y del propio lenguaje. Básicamente, la portabilidad de las interfaces creadas de esta manera serán las que proporcione el lenguaje.



La creación de interfaces de usuario usando lenguajes de descripción basados en XML pretende solventar esto, proporcionando un medio que permita construir interfaces mediante descripciones de alto nivel en los distintos aspectos de la interfaz: estructura y comportamiento, de modo que a partir de la descripción se pueda generar de forma automatizada la interfaz de usuario final. Además, este proceso permite centrar el desarrollo en las necesidades del usuario, puesto que no se realiza siguiendo las directrices marcadas por el lenguaje.

En este ámbito, se utilizará una notación de alto nivel para desarrollar la interfaz, que se almacena en un archivo aparte, en formato XML. Posteriormente se trata este archivo para obtener el código de la interfaz que se integrará en la aplicación. A este procedimiento se le denomina **mapear** la interfaz. El proceso de mapeado depende del lenguaje concreto que se esté usando.

Tras el correspondiente proceso se podrá visualizar la interfaz en un dispositivo final.



Reflexiona

El uso de tecnologías XML permite separar completamente la creación de la interfaz de la programación de la aplicación subyacente. ¿Qué ventajas crees que aporta esto? ¿Puede mejorar el procedimiento de desarrollo de aplicaciones con interfaces gráficas complejas?

Mostrar retroalimentación

De hecho, así es, utilizar tecnologías XML para la creación de interfaces tiene como principal ventaja que, después de planificar adecuadamente qué esperamos de la aplicación y qué información debe proporcionarle la interfaz, el equipo puede dividirse totalmente el trabajo de desarrollo, ya que ambas tareas pueden realizarse completamente por separado con el consiguiente ahorro de tiempo. Además, las modificaciones que se pueden realizar influyen mucho menos en la aplicación global y suelen afectar tan solo al ámbito que corresponde. Por otra parte, al utilizar lenguajes y

herramientas específicas, normalmente, se consiguen interfaces más elaboradas ya que se pone a nuestra disposición elementos concretos de este ámbito de la programación.

A continuación, veremos algunos ejemplos de lenguajes para el desarrollo de interfaces basados en XML y como realizan el proceso de mapeado.



Autoevaluación

¿Qué frase es correcta con respecto al desarrollo de interfaces?

- ☐ Es un proceso sencillo porque la misma interfaz es válida para todos los usuarios.
- ☐ Una vez creada un interfaz en un lenguaje de alto nivel podremos reutilizarla para cualquier tipo de dispositivo sin necesidad de modificarla.
- ☐ Una interfaz descrita con un lenguaje basado en XML se integra directamente en la aplicación.
- ☐ El uso de interfaces descritas con lenguajes basados en XML permite desarrollar por separado la interfaz gráfica de una aplicación y comportamiento.

No es correcta ya que el desarrollo de interfaces está íntimamente relacionado con el usuario final y debe hacerse de acuerdo a sus necesidades.

No es correcto porque la adaptación de la interfaz a diferentes dispositivos depende del lenguaje, si el lenguaje no es adecuado para, por ejemplo, un teléfono móvil no podremos usar esa interfaz en una aplicación para móviles.

No es correcta ya que las interfaces descritas con lenguajes basados en XML deben sufrir un proceso de transformación para integrarse en la aplicación final.

Correcto, se puede dividir el trabajo en dos grupos, uno que genere los documentos con todas las interfaces y otro que desarrolle toda la funcionalidad, dejando la integración para el final del proyecto.

Solución

1. Incorrecto (#answer-20_63)
2. Incorrecto (#answer-20_105)
3. Incorrecto (#answer-20_108)
4. Opción correcta (#answer-20_111)

2.2.- XAML.

Es un lenguaje de marcas empleado para la creación de interfaces en el modelo de programación .NET Framework de Microsoft. Las aplicaciones creadas podrán ejecutarse en entornos Windows.

```
<Page
  xmlns="http://schemas.microsoft.com/winfx/2006/xaml/
  presentation"
  xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
  x:Class="EspacioEjemplo.PaginaEjemplo">
  <Button Click="Accion_Click" >Haz Click!</Button>
</Page>
```

Q María José Navascués González

(Elaboración propia)

Mediante XAML y XUL se pueden crear lo que se conoce como RIA (Rich Interface Applications) que contienen abundante material gráfico. XAML consta de una serie de elementos XML para representar los principales componentes gráficos, así como la distribución, paneles y manejadores de eventos. Puede hacerse programando directamente la interfaz mediante un editor de texto, o mediante el entorno de desarrollo gráfico incluido en la herramienta Expression Blend. El código asociado a la interfaz es puramente declarativo, es decir, hace referencia tan solo al aspecto visual, pero no añade funcionalidad, salvo ciertas respuestas muy sencillas a interacciones con el usuario.

La realización de cálculos se añade en un archivo independiente. Esto permite al equipo de desarrollo y al de diseño trabajar por separado, sin interferir mutuamente en su trabajo.

Proceso de mapeado: se realiza en tiempo de ejecución, los elementos de la interfaz se conectan con objetos del framework .NET y los atributos con propiedades de estos objetos para integrarlos en la aplicación. Para facilitar la traducción de XAML a código .NET se ha creado una relación para cada elemento XAML con una clase de la plataforma .NET.

Tienes el código del ejemplo en el siguiente documento:

[Documento con ejemplo para XAML.](#)



Para saber más

Para conocer algo más sobre XAML en su página oficial puedes encontrar una introducción:

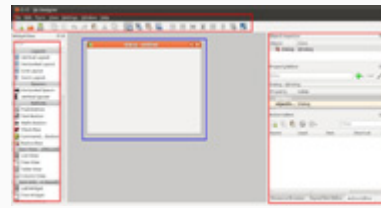
[Página oficial de XAML.](#)

Y también puedes visitar la siguiente página donde poder encontrar la referencia del lenguaje y ejemplos de aplicaciones dentro del ámbito del entorno de desarrollo Expression Blend.

[Página de la MSDN de XAML.](#)

3.- Herramientas para la creación de interfaces multiplataforma.

Existen muchos tipos de software para la creación de interfaces de usuario. Habitualmente estas herramientas tienen en común que para generar interfaces gráficas usan un sistema de ventanas, las cuales permiten la división de la pantalla en diferentes regiones rectangulares, llamadas "ventanas" cuya parte central es el conjunto de herramientas (toolkit).



Q. María José Navascués González

(Elaboración propia)

Cuando creamos una aplicación con interfaces gráficas, en primer lugar, hay que **diseñar la interfaz**, normalmente el toolkit contiene los objetos gráficos más empleados, tales como menús, botones, etiquetas, barras de scroll, y campos para entrada de texto que compondrán la interfaz, mientras que el sistema de ventanas provee de procedimientos que permiten dibujar figuras en la pantalla y sirve como medio de entrada de las acciones del usuario. En la imagen puedes apreciar los conjuntos de herramientas enmarcados en rojo (en la imagen cuadro derecho y barra de herramientas superior) y la zona de dibujo en azul (en la imagen cuadro central). En algunas aplicaciones denominan **Widgets** a los controles o elementos que podemos añadir a una interfaz.

A continuación, se **añade funcionalidad** a los elementos de la interfaz a través de una serie de procedimientos definidos por el programador o programadora. La función de estos procedimientos es el decidir la forma en que se comportarán los objetos gráficos.

Por último, se **conecta la interfaz generada con la aplicación de destino**. Esto se puede realizar de diferentes maneras. Si usamos un único lenguaje de programación para la interfaz y la funcionalidad, lo usual es contar con un entorno de desarrollo integrado común para todas las fases de desarrollo como al utilizar la biblioteca swing con Java. Sin embargo, cuando el lenguaje final es uno y el lenguaje de la interfaz otro, como es el caso de las tecnologías basadas en XML que estamos viendo, normalmente se requiere de un proceso de **traducción** de los elementos XML a elementos que entienda la plataforma final, como en el caso de XAML que traduce cada elemento XML a clases de la plataforma .NET. En el caso de XUL es el motor de renderización el que se encarga de hacer las transformaciones necesarias para visualizar la interfaz. Este proceso puede ser más o menos automático en función de las tecnologías seleccionadas.

A continuación, veremos algunos ejemplos de herramientas existentes, tanto libres como propietarias, que difieren en la manera que tratan los archivos XML con las interfaces para incluirlos en la aplicación final.



Texto de la pregunta: Las herramientas para la generación de interfaces...

- ☐ ...son muy complicadas de aprender.
- ☐ ...no se integran nunca con otras herramientas de desarrollo de aplicaciones.
- ☐ ...se usan para editar la interfaz y poder modificarla en modo texto.
- ☐ ...permiten al programador o programadora concentrarse en los aspectos relativos al diseño.

No es correcta, aprender a manejar estas herramientas suele ser un proceso intuitivo y bastante sencillo debido a su estructura de ventanas y a que la construcción de la interfaz se basa en el procedimiento arrastrar y soltar.

No es correcto, hoy día encontramos entornos integrados de desarrollo que incorporan todas las herramientas necesarias para la construcción de aplicaciones en todas las fases de desarrollo.

No es correcto, el uso habitual de estas herramientas es en modo gráfico.

Correcta, al abstraernos de la sintaxis y aspectos internos de cada lenguaje de modelado conseguimos centrarnos en el diseño de la interfaz.

Solución

1. Incorrecto (#answer-34_63)
2. Incorrecto (#answer-34_137)
3. Incorrecto (#answer-34_140)
4. Opción correcta (#answer-34_143)

3.1.- Presentación de algunas herramientas.

Las herramientas de diseño de interfaces gráficas de usuario forman parte de las herramientas RAD (Rapid Application Development, Desarrollo rápido de Aplicaciones).



Q Ministerio de Educación y

Formación Profesional. (Elaboración propia)

El **principal objetivo** de estas herramientas es ocultar la sintaxis de los lenguajes de modelado y proporcionarles una interfaz que permita especificar adecuadamente el modelo de interfaz en los tres aspectos que hemos visto, a saber:

La interfaz se almacena en un archivo de texto plano siguiendo las directrices del estándar XML.

Estas herramientas, disponen de editores, intérpretes, generadores y otras aplicaciones útiles para llevar a cabo tareas relacionadas con la elaboración, manipulación o generación de modelos de interfaz.

Entre otras, con estas características podemos encontrar las siguientes aplicaciones:

■ Libres:

- QT Designer.
- Glade.
- Scene Builder

■ Propietarias:

- Expression Blend de Microsoft.
- Flex de Adobe.

Las aplicaciones RIA (Rich Internet application), "aplicación de Internet enriquecida" o "aplicación rica de internet", es una aplicación web que tiene la mayoría de las características de las aplicaciones de escritorio



Para saber más

Si tienes curiosidad por saber algo más de estas herramientas aquí tienes los enlaces que te llevarán a sus páginas oficiales:

[Página oficial de QT Creator.](#)

[Página oficial de GLADE.](#)

[Página oficial de FLEX.](#)

[Página oficial de Microsoft Expression Blend.](#)

[Diseñar XAML en Visual Studio y Blend para Visual Studio](#)



Autoevaluación

Texto de la pregunta: ¿Qué es una aplicación RIA?

- ☐ Aquella que solo ejecuta en un terminal.
- ☐ Un programa con una interfaz de usuario compleja.
- ☐ Una aplicación para realizar operaciones bancarias.
- ☐ Aquella que se ejecuta en un navegador web y su aspecto y funcionalidad es el de una aplicación de escritorio.

No es correcta, no tiene nada que ver.

No es correcto, no son programas independientes.

No es correcto, te has despistado.

Correcta, por eso es tan importante la creación de interfaces multiplataforma.

Solución

1. Incorrecto (#answer-39_63)
2. Incorrecto (#answer-39_151)
3. Incorrecto (#answer-39_154)
4. Opción correcta (#answer-39_157)

3.2.- Diseñador VS vs Blend para VS

Característica	Diseñador de Visual Studio	Blend for Visual Studio
Enfoque principal	Escritura de código, lógica de negocio y depuración.	Diseño visual, estética y experiencia de usuario (UX).
Animaciones	Edición manual limitada en el código XAML.	Línea de tiempo completa para crear y editar animaciones y storyboards.
Estados visuales	No tiene herramientas nativas potentes para gestionarlos.	Herramientas específicas para diseñar y probar Visual States y transiciones.
Recursos y estilos	Gestión básica de diccionarios de recursos.	Capacidad avanzada para extraer y modificar plantillas de control (Control Templates) con un clic.
Interactividad	Requiere programar eventos en C# o VB.	Uso de Behaviors (comportamientos) para añadir interactividad sin escribir código.
Datos de prueba	Carga de datos reales durante la ejecución.	Generación de datos de muestra para previsualizar la interfaz sin conectarse a una base de datos real.



Reflexiona: Diseñador de Visual Studio vs Blend

Ambas herramientas pueden trabajar de forma simultánea sobre el mismo proyecto. Si realizas un cambio en Blend y guardas, Visual Studio te pedirá recargar el archivo para mostrar la actualización y viceversa. Comparten el mismo motor y formato de archivo (XAML), pero están diseñados para perfiles y tareas diferentes. Mientras que Visual Studio se centra en la lógica y el código, Blend es una herramienta especializada en el diseño visual avanzado y la experiencia de usuario.

- Diseñador XAML (Visual Studio): Está enfocado principalmente en el

desarrollador. Ofrece una experiencia equilibrada entre la escritura de código y el diseño visual básico, como arrastrar controles y ajustar propiedades rápidas.

- Blend for Visual Studio: Es una herramienta especializada para el diseño avanzado de interfaces de usuario. Se incluye con Visual Studio y se puede abrir por separado o desde el mismo IDE haciendo clic derecho sobre un archivo XAML y seleccionando "Diseñar en Blend".



El problema del XAML auto-generado

Es muy tentador usar el diseñador visual, arrastrar controles, hacer clic derecho y darle a "Agrupar". Parece que funciona como PowerPoint o como el antiguo Windows Forms. Pero realmente usando de forma nativa el framework WPF:

Usar el diseñador para estructurar la interfaz de usuario de la aplicación o componente va a empeorar la responsividad del código XAML

Mejor escribir el XAML a mano y usar la vista de diseño solo como un "monitor" para ver cómo va quedando.

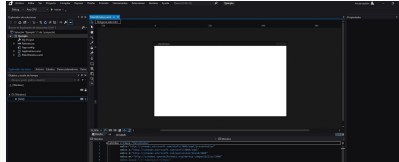
Por ejemplo: un botón centrado. Lo que escribimos a mano sería así de sencillo:

```
<Button Content="Enviar" HorizontalAlignment="Center" Padding="10">
```

4-. Blend para Visual Studio.

Blend para Visual Studio nos permite construir el diseño de aplicaciones para la Tienda de Windows, Windows Phone, WPF y Silverlight. Actualmente se instala junto con Visual Studio.

Para comenzar a trabajar, necesitaremos crear un proyecto. Para ello, podemos utilizar plantillas existentes que nos facilitan el diseño de la aplicación.

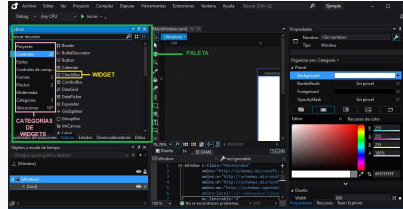


Q Montaña Martín Vergel (Elaboración propia)

4.1.- Controles. Contenedores.

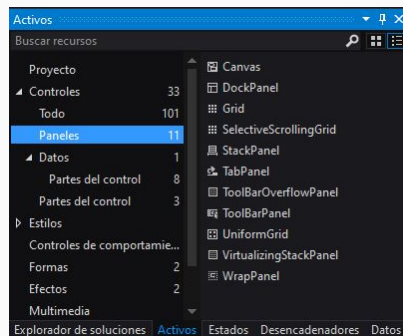
Los controles que podemos utilizar los podemos encontrar en la Paleta. Se muestra en el lado izquierdo y organiza los widgets por categorías. Las categorías que podemos encontrar son:

Proyecto, Controles, Estilos, Controles de componentes, Formas, Efectos, Multimedia, Categorías y Ubicaciones.



Q Montaña Martín Vergel (Elaboración propia)

Los widgets contenedores están situados en la paleta dentro del menú Controles -> Paneles

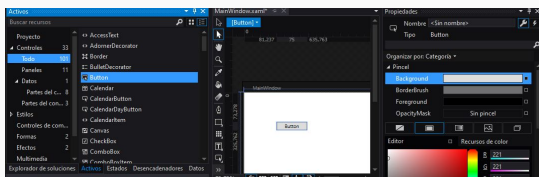


Q Montaña Martín Vergel (Elaboración propia)

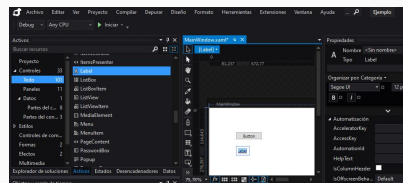
4.2.- Controles y propiedades.

Los controles más utilizados en Blend Studio son: Cuadros de textos o entrada de texto, etiquetas, botones, opciones, listas desplegables y casillas de verificación. Todo esto debe resultarnos familiar, pero vamos a realizar un pequeño recordatorio:

- Entrada de texto (Text Entry o TextBox): nos permiten introducir datos en nuestras aplicaciones, aunque también es posible mostrar información a los usuarios en tiempo de ejecución a través de ellos.
- Etiqueta (Label): sirven para mostrar por pantalla texto.
- Botón (Button): nos permite que la aplicación ejecute un conjunto de acciones cuando el usuario los seleccione.
- Opciones (Radio /RadioButton): permite indicar varias opciones para que el usuario solo seleccione una.
- Casillas de verificación: permite presentar una o varias opciones. El usuario podrá seleccionar una o varias con independencia de la que seleccionemos.
- Lista desplegable: nos permite mostrar un conjunto de valores para que el usuario seleccione uno de ellos.

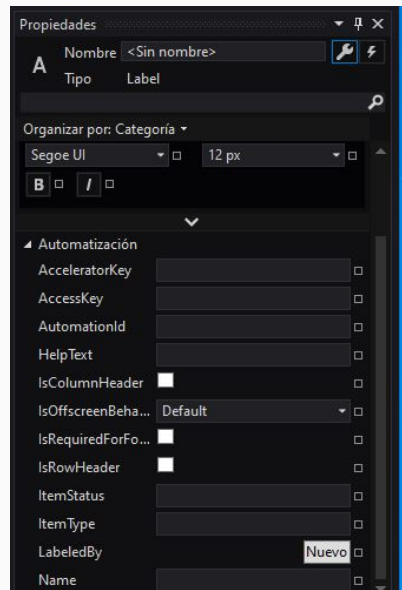


Q Montaña Martín Vergel (Elaboración propia)



Q Montaña Martín Vergel (Elaboración propia)

La mayoría de los controles, tienen una serie de propiedades: el color, la ubicación, los estilos que se le aplican, el tamaño, etc.



Q Montaña Martín Vergel (Elaboración propia)

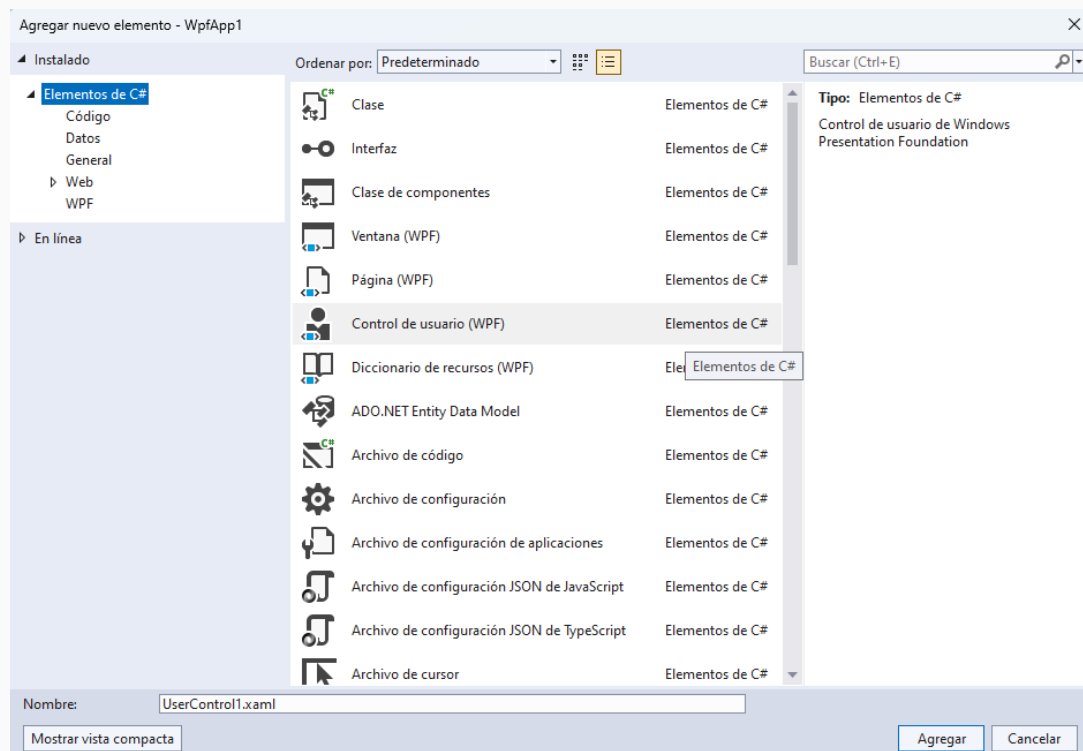
4.3.- Controles personalizados

Crear tus propios controles en WPF te permite encapsular lógica, estilo y funcionalidad personalizada en un solo elemento reutilizable. Puedes hacerlo mediante User Controls o Custom Controls.

Métodos para crear controles personalizados

■ User Control:

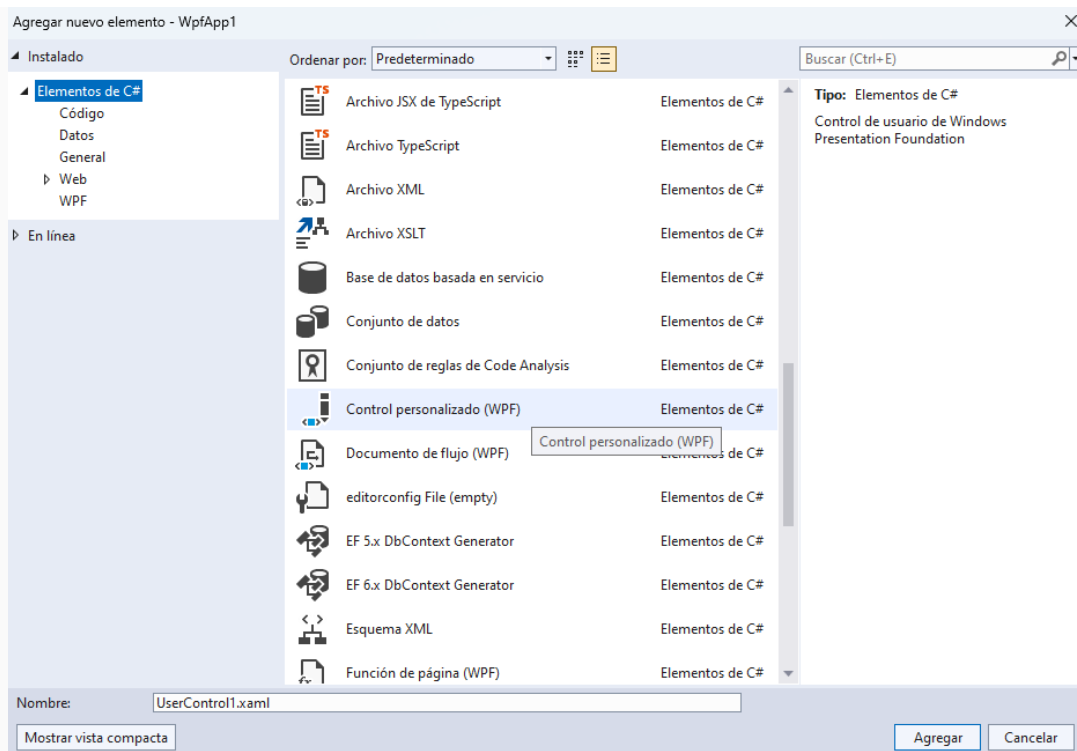
- Más sencillo.
- Combina otros controles existentes para crear un nuevo control reutilizable.



Q Sandra Alonso Díaz (Elaboración propia)

■ Control personalizado:

- Más avanzado.
- Diseñado desde cero, ideal para necesidades específicas o para personalización profunda.



Q Sandra Alonso Díaz (Elaboración propia)

5.- Análisis y edición de documentos XML.

Siempre que trabajemos con documentos XML es conveniente su validación. La validación consiste en la comprobación de que un documento XML se encuentre bien formado y se ajusta a una estructura bien definida. Un documento bien formado sigue las reglas básicas de XML establecidas para el diseño de documentos. Un documento válido, además, respecta las normas indicadas en el fichero DTD (definición tipo de documento) al que se debe de encontrar asociado.

La validación consiste en:

- 1-. La corrección de los datos: permite detectar valores nulos o fuera de rangos.
- 2-. La integridad de los datos: se comprueba que toda la información obligatoria se encuentre presente en el interior del documento.
- 3-. El entendimiento compartido de los datos: se comprueba que el emisor y el receptor van a recibir el documento de la misma forma y que lo puedan interpretar igual.

Para la creación de documentos XML existen herramientas que evitan que se introduzcan datos erróneos, errores tipográficos, sintácticos y de contenidos. Estas herramientas se conocen con el nombre de Editores XML.

Los editores de XML tienen que ser capaces de:

- 1-. Leer la DTD correspondiente al documento y presentar una lista desplegable con los elementos disponibles enumerados en la DTD, evitando la inclusión de algún elemento incluido en el esquema.
- 2-. Advertir del olvido de una etiqueta obligatoria e incluso no permitir este tipo de descuido o errores, No permite la finalización del documento si existen errores.



Debes conocer

Existen editores de XML que ocultan el código del documento y lo presentan en un formato más fácil de utilizar.

Ejemplos de editores XML son:

Gratuitas:

XMLPad de WMHelp. Lo puedes descargar desde el siguiente enlace: [XMLPAD](#)

XML Copy Editor. Posee licencia GNU. Lo puedes descargar desde el siguiente enlace: [XML Copy Editor](#)

XML Explorer: Lo puedes descargar desde el siguiente enlace: [XMLExplorer](#)

Versiones gratuitas de software comercial con periodos de evaluación:

XMLSpy. Lo puedes descargar desde el siguiente enlace:

Altova XMLSpy

Editix XML Editor. Lo puedes descargar desde el siguiente enlace:

 [Editix XML Editor](#)

File Viewer Plus 3. Lo puedes descargar desde el siguiente enlace:

[File Viewer Plus 3](#)

6.- Código para diferentes plataformas.

Los documentos XML son procesados a través de analizadores que leen el documento, lo interpretan y genera una salida en base a su contenido. El resultado lo mostrará, por ejemplo, en un dispositivo, en una ventana de un navegador, en una impresora, etc.

Se denomina **parsing o análisis-sintáctico** al hecho de que una aplicación reciba un fichero XML, lo cargue en memoria y lo procese para obtener unos resultados. A este tipo de aplicaciones se les denomina parsers o analizadores (léxicos-sintácticos).

Se dice que el **parsing XML** es el proceso mediante el cual se lee y se analiza un documento XML para comprobar que está bien formado para pasar su contenido a una segunda aplicación para su procesamiento,

Existen dos tipos de analizadores XML:

DOM (parsers DOM): es un metalenguaje con una estructura jerárquica. Las marcas dentro del documento tienen una relación padre-hijo. Es decir, posee una estructura en forma de árbol. Cada uno de los elementos del documento recibe el nombre de nodo, Estos nodos serán: elementos, atributos, comentarios, etc.

SAX: el API de SAX especifica como ha de comportarse el parser, es decir, especifica interfaces de programación y no clases. SAX se encuentra compuesto por dos interfaces: el XMLReader que representa el parser y el ContentHandler que recibe los datos del parser.

7.- Introducción WPF

WPF son las siglas de Windows Presentation Foundation, y es un framework de Microsoft para desarrollar aplicaciones de escritorio en Windows. Se introdujo con .NET Framework 3.0 como una alternativa más moderna y potente a Windows Forms, y se utiliza para crear interfaces gráficas de usuario (GUI) ricas y flexibles.

■ ¿Qué es WPF? Fuente: learn.microsoft.com.

Características clave de WPF:

Soporte para gráficos avanzados

Utiliza DirectX, lo que permite renderizar gráficos 2D y 3D de manera eficiente. Soporta efectos visuales avanzados como sombras, transparencias y gradientes.

Diseño declarativo con XAML

Utiliza XAML (Extensible Application Markup Language) para definir la interfaz de usuario de manera declarativa.

La separación entre diseño (XAML) y lógica (C# o VB.NET) facilita el desarrollo colaborativo entre diseñadores y programadores.

Data Binding (Enlace de datos)

Permite conectar la interfaz de usuario con datos o propiedades en el código, mejorando la sincronización y reduciendo la necesidad de código repetitivo.

Soporte para multimedia

Permite integrar fácilmente imágenes, audio y video en las aplicaciones.

Escalabilidad y DPI-awareness

Diseñado para manejar diferentes resoluciones y configuraciones de DPI, asegurando que las aplicaciones se vean bien en cualquier pantalla.

Compatibilidad con animaciones

Incluye un sistema de animación robusto para crear transiciones y efectos visuales atractivos.

Comparativa: Ventajas e inconvenientes de Windows Forms y WPF:

Aspecto	Windows Forms	WPF (Windows Presentation Foundation)
Curva de aprendizaje	✅ Más sencilla, ideal para principiantes.	⚠️ Más compleja debido a XAML y conceptos avanzados como MVVM.
Gráficos y diseño	⚠️ Limitado: Basado en GDI+, con soporte básico para gráficos y personalización.	✅ Potente: Soporte para gráficos avanzados, animaciones y efectos visuales.
Flexibilidad	⚠️ Menos flexible: Controles rígidos y personalización limitada.	✅ Muy flexible: Controles personalizables y diseño completamente modular.
Compatibilidad	✅ Compatible con aplicaciones heredadas y tecnologías antiguas.	⚠️ Menos adecuado para integraciones con proyectos antiguos basados en Windows Forms.
Rendimiento	⚠️ Menor rendimiento en interfaces complejas o aplicaciones gráficas intensivas.	✅ Excelente rendimiento gracias a DirectX para gráficos y procesamiento visual.
Escalabilidad/DPI	⚠️ Mal soporte para pantallas de alta resolución y diferentes configuraciones DPI.	✅ Escalable: Se adapta automáticamente a resoluciones y DPI diferentes.
Soporte multimedia	⚠️ Muy básico, no ideal para integrar videos o animaciones avanzadas.	✅ Excelente soporte para multimedia (audio, video, animaciones).
Herramientas de diseño	⚠️ Dependencia del diseñador básico en Visual Studio.	✅ Herramientas avanzadas como Blend para Visual Studio.
Ecosistema de controles	✅ Amplia colección de controles nativos y bibliotecas de terceros ya disponibles.	✅ Colección moderna de controles con capacidades avanzadas.
Separación de lógica y diseño	⚠️ No está diseñado para separación clara entre lógica y diseño.	✅ Compatible con MVVM: Separación clara entre lógica, datos y diseño.
Uso recomendado	✅ Aplicaciones simples y empresariales con lógica sencilla.	✅ Aplicaciones modernas, gráficas intensivas, y con interfaces ricas.

Q Licencia: Dominio público

¿Por qué WinForms no desaparece?

- **Velocidad de desarrollo:** Para utilidades internas sencillas o aplicaciones de base de datos donde la estética no importa, WinForms sigue siendo más rápido de implementar.
- **Mantenimiento:** Existen millones de aplicaciones empresariales críticas escritas en WinForms que no necesitan ser migradas porque siguen cumpliendo su función.
- **Rendimiento en hardware antiguo:** Al ser más ligero, WinForms puede responder mejor en equipos con recursos muy limitados.

Así, podemos concluir que Windows Forms es mejor para proyectos simples, rápidos y compatibles con tecnologías antiguas y WPF es ideal para aplicaciones modernas que requieren interfaces avanzadas, gráficos ricos y un diseño escalable.

Actualmente, el verdadero "sucesor" que Microsoft intenta impulsar para

interfaces aún más modernas es
WinUI 3, pero WPF sigue siendo la opción más estable y madura para el
desarrollo de software de escritorio de Windows

.

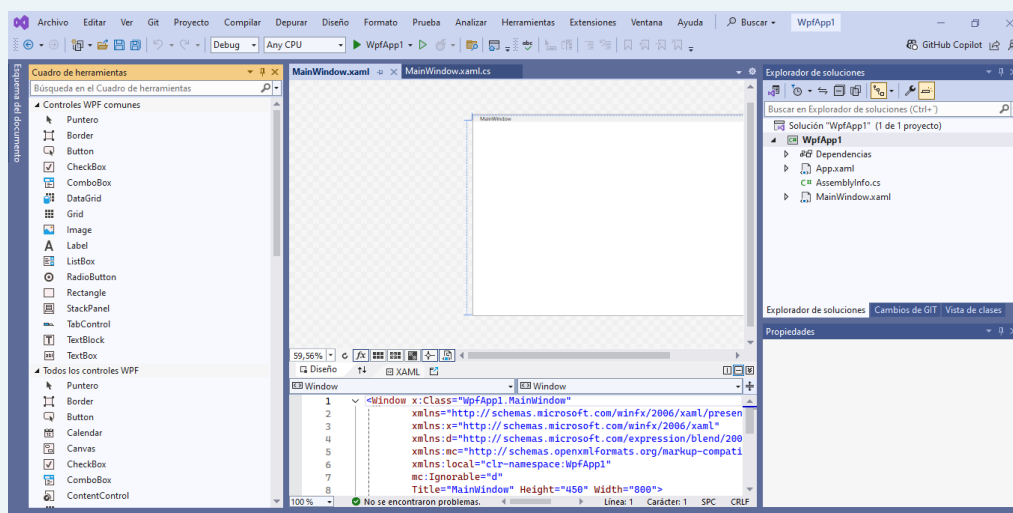
7.1.-Ejemplo de desarrollo de una interfaz utilizando WPF.

Debemos tener en cuenta que Blend for Visual Studio es una aplicación de diseño visual que permite a los desarrolladores diseñar y personalizar interfaces de usuario para aplicaciones WPF de manera eficiente y sin necesidad de codificación directa. Los usuarios pueden arrastrar y soltar controles en un área de diseño, modificar sus propiedades visualmente, aplicar estilos y plantillas, y crear animaciones interactivas. El resultado es un archivo XAML, que define la interfaz gráfica, y puede integrarse directamente en un proyecto WPF, permitiendo vincular la interfaz de usuario a la lógica de la aplicación mediante patrones como MVVM.



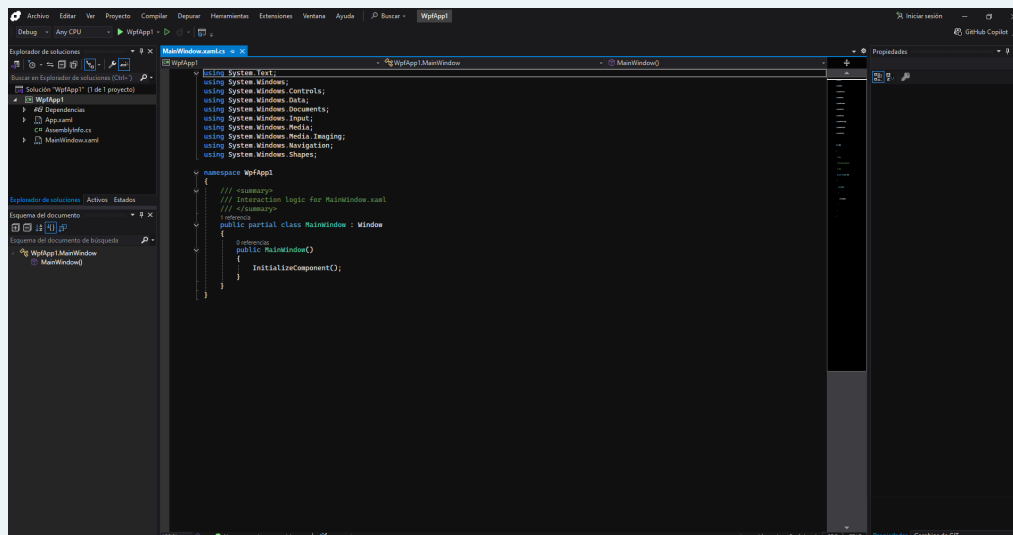
Debes conocer

Para utilizar Blend for Visual Studio es necesario tenerlo instalado en el equipo. Este software se incluye como parte de la instalación de Visual Studio, por lo que no requiere una descarga separada. Solo asegúrate de que la opción "Desarrollo de aplicaciones de escritorio con .NET" esté habilitada durante la instalación. Una vez configurado, podrás acceder a Blend desde el menú de Visual Studio.



 Sandra Alonso Díaz (Elaboración propia)

Para editar los archivos XAML utilizando Blend for Visual Studio desde un proyecto en Visual Studio, no es necesario realizar configuraciones adicionales, ya que Blend se integra perfectamente con Visual Studio. Solo debes abrir el archivo XAML desde el explorador de soluciones haciendo clic derecho sobre él y seleccionando la opción Abrir en Blend. Esto abrirá el archivo en el entorno de diseño de Blend, en caso de que no lo tengas ya y podrás modificar visualmente la interfaz y guardar los cambios, los cuales se reflejarán directamente en el proyecto.



 Sandra Alonso Díaz (Elaboración propia)

a.- Descripción del problema.

En primer lugar, hay que destacar que en estos materiales se pretende dar un vistazo rápido al funcionamiento de una herramienta para la creación de interfaces de usuario usando XML, así como la posterior integración de esta interfaz en una aplicación. Por lo tanto, no podrá considerarse esta unidad como un manual completo de aprendizaje de Blend para Visual Studio, sino como ejemplo didáctico de la materia que se pretende trasladar al alumnado.

Para ilustrar el uso de la herramienta Blend para Visual Studio y la inclusión de los archivos de interfaz generados en una aplicación WPF vamos a volver a desarrollar nuestra aplicación de **gestión de una cadena hotelera**.

No vamos a implementar la funcionalidad subyacente a la interfaz, es decir, añadir la reserva a la base de datos del hotel, esto se deja para otros módulos del ciclo.



Q [Alex Coiro](#) (GNU/GPL)



Reflexiona

Las interfaces de ejemplo son muy genéricas. Ten en cuenta que según el tratamiento que demos finalmente a la interfaz podremos obtener una aplicación de escritorio tradicional, una aplicación web o una aplicación para móviles. Sin embargo, la interfaz a generar será siempre la misma y podremos usar el mismo archivo XML en los tres casos.

b.- Creación proyecto WPF.

Comenzaremos por crear en Visual Studio un nuevo proyecto WPF. Para ello, seleccionamos Archivo > Nuevo proyecto y, en las plantillas disponibles, buscamos y seleccionamos Aplicación WPF. Una vez creada, Visual Studio generará automáticamente los archivos básicos, incluido un archivo XAML (MainWindow.xaml), que podrás personalizar tanto en Visual Studio como en Blend. Pondremos como nombre de proyecto MiHotelWPF

Si observamos los archivos generados al crear un proyecto WPF en Visual Studio, veremos que se han creado varios archivos clave, entre los que destaca un archivo con extensión .xaml. Este archivo contiene código XAML para definir la interfaz gráfica de la aplicación. Al abrirlo, podremos ver que se ha generado un diseño básico que incluye un contenedor (Grid) preparado para albergar controles. Por defecto, este contenedor está vacío, listo para que agregues botones, cuadros de texto u otros elementos para construir tu interfaz.

c.- Diseño de la interfaz.

Comenzaremos a crear la interfaz accediendo a Blend for Visual Studio, que se abrirá automáticamente cuando seleccionemos la opción Abrir en Blend en el menú contextual del archivo MainWindow.xaml generado en el apartado anterior.

Para empezar, ampliaremos el contenedor principal (Grid) que aparece por defecto y eliminaremos cualquier elemento que se haya insertado automáticamente.

A continuación, agregaremos los textos y cuadros de texto necesarios para diseñar la interfaz. Utilizaremos controles TextBlock para mostrar los textos como etiquetas, y TextBox para la entrada de datos. La interfaz incluirá etiquetas para los siguientes campos:

Título: "Datos del Cliente".

Información del cliente: "Nombre y apellidos", "DNI", "Dirección", "Localidad", "Provincia".

Título secundario: "Datos de la reserva".

Información de la reserva: "Fecha de llegada", "Fecha de salida", "Número de habitaciones", "Tipo de habitación", "Régimen", y un texto informativo sobre la selección de una habitación para fumador.

Todos estos elementos se pueden organizar fácilmente en el contenedor Grid utilizando filas y columnas para un diseño limpio y profesional.

Arrastraremos cada TextBlock a nuestra interfaz y los colocaremos en el lugar correspondiente dentro del contenedor principal (Grid). Podemos crear un contenedor para cada apartado y cambiar las propiedades para hacer cambios en el diseño.

Una vez insertados en el diseño, accederemos al Panel de Propiedades para personalizar cada etiqueta y lograr el aspecto deseado. En este panel, modificaremos las siguientes propiedades según sea necesario:

- Text: Para establecer el texto que queremos mostrar en cada etiqueta.
- FontSize: Para ajustar el tamaño de la fuente.
- FontWeight: Para aplicar estilos como "Bold" (negrita).
- Foreground: Para cambiar el color del texto.

Este proceso se repetirá para todas las etiquetas, asegurándonos de alinear y distribuir los elementos correctamente en la interfaz.

El siguiente paso es insertar los cuadros de texto. Elegiremos el objeto TextBox. Arrastraremos un TextBox a nuestra interfaz y lo colocaremos en el lugar correspondiente dentro del contenedor principal (Grid).

Una vez insertados, utilizaremos el Panel de Propiedades para personalizar cada cuadro de texto y adaptarlo al diseño deseado. En este panel podremos modificar:

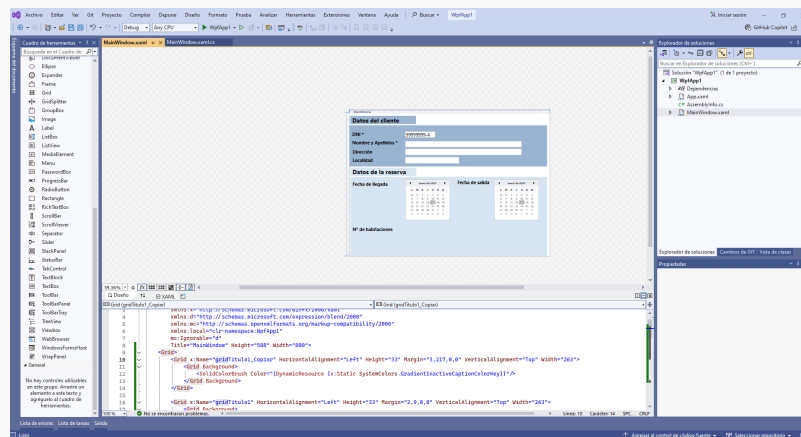
- Width y Height: Para ajustar el tamaño del cuadro de texto.
- Text: Para establecer texto de marcador de posición si es necesario.

- **FontSize:** Para definir el tamaño del texto ingresado.
- **Margin:** Para establecer el espaciado alrededor del cuadro de texto.

Este procedimiento se repetirá para cada cuadro de texto requerido en la interfaz, alineándolos correctamente junto a las etiquetas correspondientes.

El siguiente paso es añadir los controles necesarios para que los usuarios puedan seleccionar una fecha mediante un calendario. En Blend for Visual Studio, podemos encontrar el control DatePicker o el Calendar. Arrastraremos el control escogido a la interfaz y lo colocaremos en el lugar correspondiente dentro del contenedor principal (Grid).

Para introducir el número de personas, utilizaremos nuevamente el control TextBox, que podemos encontrar en la categoría Controls. Arrastraremos un TextBox a la interfaz y ajustaremos su tamaño y propiedades desde el Panel de Propiedades.



Q Sandra Alonso Díaz (Elaboración propia)

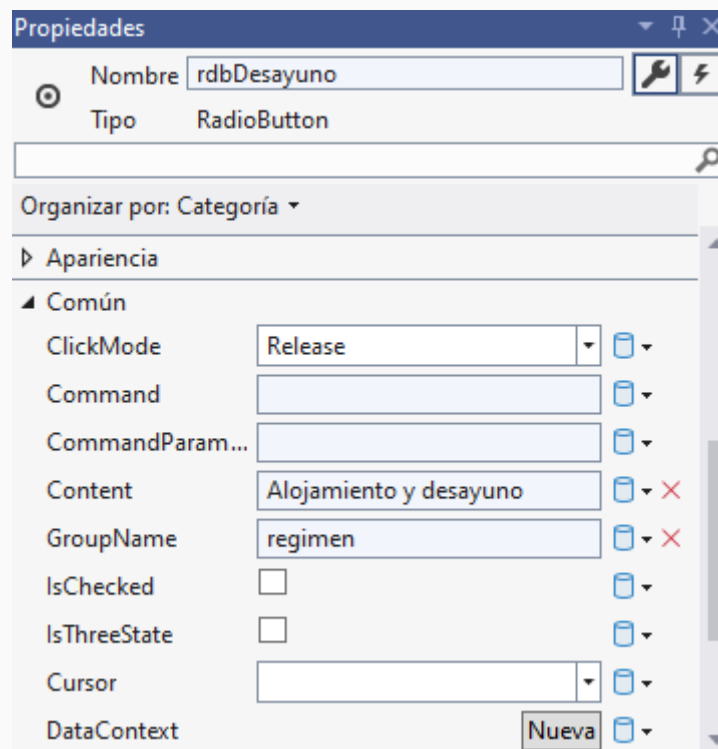
Para indicar el tipo de habitación, emplearemos un ComboBox. Este control también se encuentra en la categoría Controls. Arrastraremos el ComboBox a la interfaz y lo colocaremos en el lugar deseado. Los valores que aparecerán en el ComboBox no se pueden definir directamente desde Blend, por lo que se deben configurar mediante código en el archivo MainWindow.xaml.cs.

En el constructor de la ventana principal (MainWindow.xaml.cs), agregaremos los valores al ComboBox:

```
0 referencias
public MainWindow()
{
    InitializeComponent();
    TipoHabitacionComboBox.Items.Add("Individual");
    TipoHabitacionComboBox.Items.Add("Doble");
    TipoHabitacionComboBox.Items.Add("Suite");
}
```

Q Sandra Alonso Díaz (Elaboración propia)

Para indicar el tipo de régimen de alojamiento, utilizaremos controles Radio Button. Al ser varias opciones, cuando se pulse alguna de ellas se tiene que desactivar las otras. Para ello, deberemos de modificar la propiedad **GroupName** a todos los radios para que contengan el mismo nombre de grupo.



Q Sandra Alonso Díaz (Elaboración propia)

Para que quepan todos los controles, tendremos que agrandar la ventana del diálogo, se agranda arrastrando con el ratón desde los bordes de la ventana. Para mover los botones basta con seleccionarlos con el ratón y arrastrarlos.

Para introducir los botones Aceptar y Cancelar, utilizaremos **Controls** de tipo **Button**.

La interfaz resultante podría ser similar a:

The image shows a user interface for a reservation system. It has a title bar 'MainWindow'. The main content is divided into two sections: 'Datos del cliente' (Client Data) and 'Datos de la reserva' (Reservation Data). The 'Datos del cliente' section has four text input fields: 'DNI *' (with the value '99999999-A'), 'Nombre y Apellidos *', 'Dirección', and 'Localidad'. The 'Datos de la reserva' section has two date pickers: 'Fecha de llegada' (Arrival Date) and 'Fecha de salida' (Departure Date). Both date pickers show a calendar for January 2025. The 'Fecha de llegada' calendar has the 17th selected. The 'Fecha de salida' calendar has the 18th selected.

Nº de habitaciones

Tipo de habitaciones

Régimen de alojamiento

- ☐ Alojamiento y desayuno
- ☐ Media Pensión
- ☒ Pensión Completa

☐ **Fumador**

En virtud de la ley de sanidad se informa a los clientes de que solo podrán fumar en habitaciones habilitadas a tal fin

Q Sandra Alonso Díaz (Elaboración propia)

d.- Funcionalidad CheckBox Fumador.

Necesitamos que el texto *"En virtud de la ley de sanidad se informa a los clientes de que solo podrán fumar en las habitaciones reservadas para tal fin"* se muestre cuando se encuentre activado el checkBox asociado al valor Fumador.

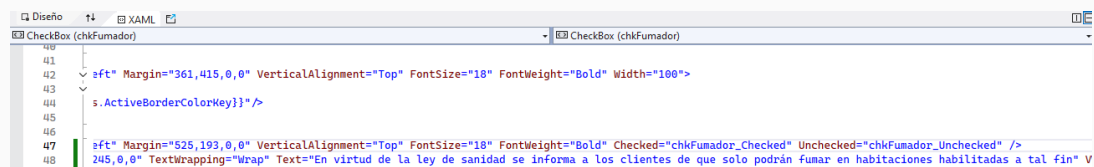
Para ello, tenemos que seguir los siguientes pasos:

Nos aseguramos de que el TextBlock (lbFumador) no esté visible al arrancar la aplicación. Para ello establecemos la propiedad Visibility a Collapsed en XAML o en el código detrás. Esto significa que el control no ocupará espacio en el diseño inicial, pero estará listo para ser visible cuando el CheckBox se active. Esto lo podemos hacer estableciendo la propiedad "Visibility" del TextBlock a "Collapsed" en el archivo XAML.



Q Sandra Alonso Díaz (Elaboración propia)

Nos quedará activar la visibilidad mediante un evento en el código detrás cuando el checkbox esté marcado. Para ello necesitaremos configurar el evento Checked y Unchecked del CheckBox. En el archivo XAML, agregamos los eventos Checked y Unchecked al CheckBox:



Q Sandra Alonso Díaz (Elaboración propia)

Ahora ya podremos manejar los eventos en el archivo .xaml.cs. En el código, definimos la lógica para mostrar u ocultar el TextBlock:

```

0 referencias
public MainWindow()
{
    InitializeComponent();
    TipoHabitacionComboBox.Items.Add("Individual");
    TipoHabitacionComboBox.Items.Add("Doble");
    TipoHabitacionComboBox.Items.Add("Suite");
}

0 referencias
private void chkFumador_Checked(object sender, RoutedEventArgs e)
{
    lblFumador.Visibility = Visibility.Visible;
}

0 referencias
private void chkFumador_Unchecked(object sender, RoutedEventArgs e)
{
    lblFumador.Visibility = Visibility.Collapsed;
}
}

```

Q Sandra Alonso Díaz (Elaboración propia)

d.- Ejemplo de creación de un control de usuario

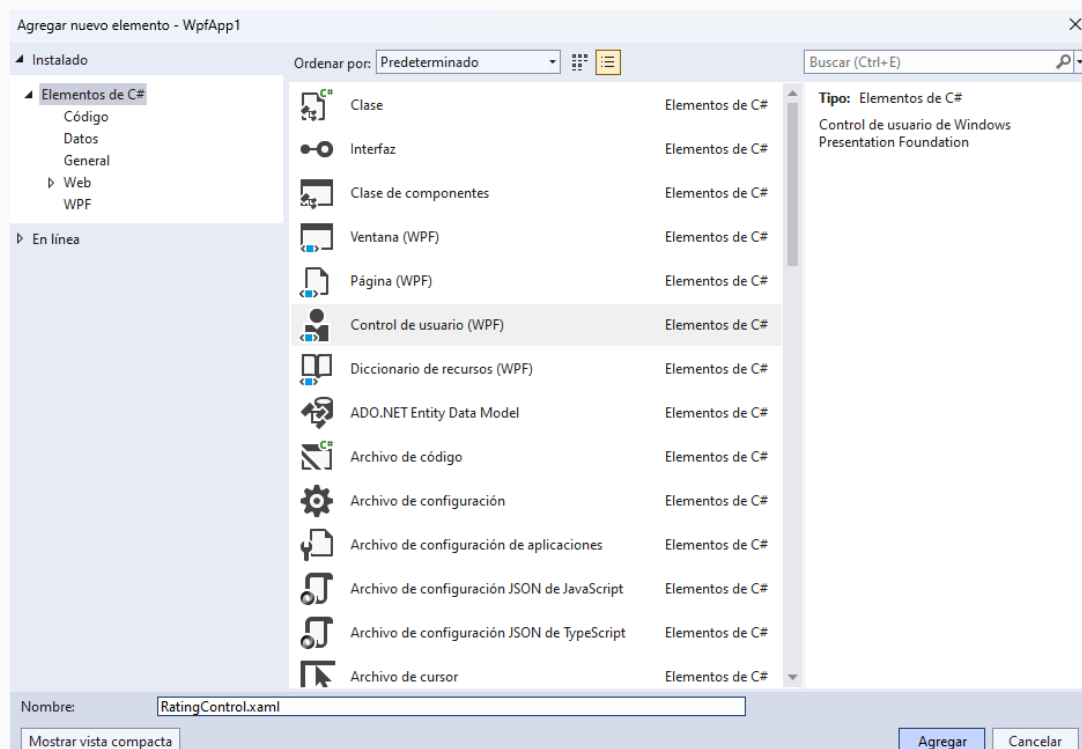
Ejemplo práctico: Crear un control de "Rating" (clasificación por estrellas)

Un Control de usuario es un conjunto de controles agrupados en un componente reutilizable.

Paso 1. Crear el Control de usuario

En Visual Studio:

- Haz clic derecho en el proyecto > Agregar > Nuevo elemento.
- Selecciona Control de usuario (WPF).
- Nómbralo, por ejemplo: RatingControl.xaml.



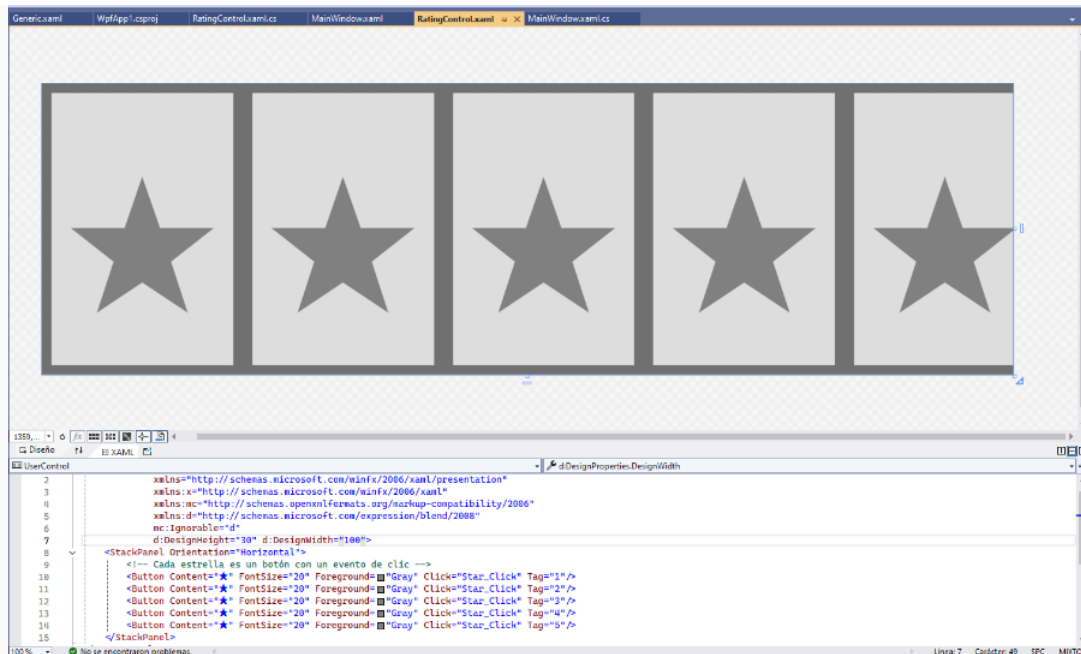
Q Sandra Alonso Díaz (Elaboración propia)

Visual Studio creará:

- RatingControl: Define la interfaz.
- RatingControl.xaml.cs: Contiene la lógica.

Paso 2. Diseñar el control (XAML)

Abre el archivo RatingControl.xaml y diseña el control. En este caso, usaremos cinco estrellas que cambian de color al seleccionarlás. Aquí tienes una muestra del código:

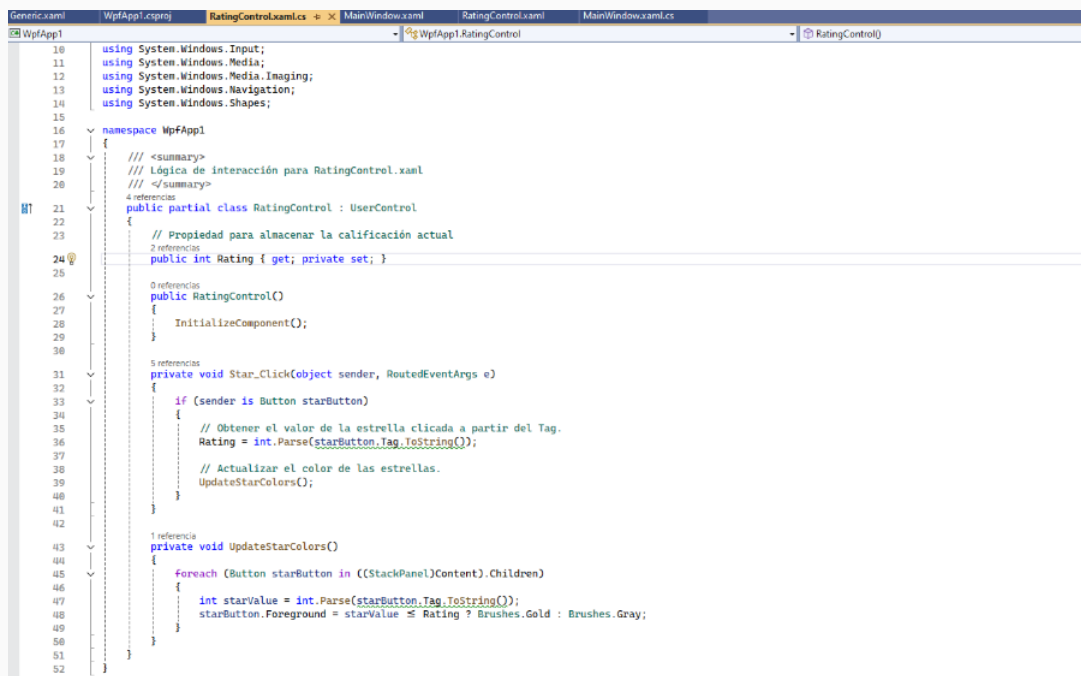


Q Sandra Alonso Díaz (Elaboración propia)

Paso 3. Agregar lógica en el archivo RatingControl.xaml.cs

Definimos una propiedad para almacenar el valor de la calificación.

Cambiamos el color de las estrellas cuando el usuario haga clic.



Q Sandra Alonso Díaz (Elaboración propia)

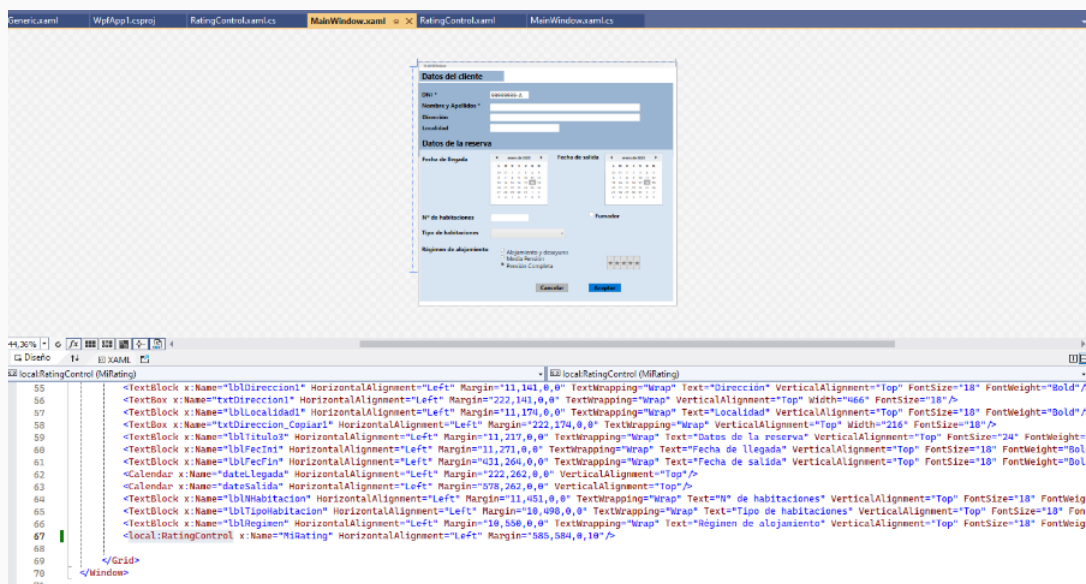
Conceptos clave

- **DependencyProperty (ValueProperty):** Es el corazón del control. A diferencia de una propiedad normal, esta permite que WPF use Data Binding. Si en tu aplicación principal tienes una base de datos con un rating, puedes enlazarla directamente a esta propiedad.

- **PropertyMetadata:** Aquí le decimos al control que, cada vez que el número cambie (ya sea por código o por clic), llame al método `OnValueChanged` para refrescar los colores de las estrellas.
- **ControlTemplate:** En el XAML, podemos usarlo para quitar al `Button` o mejor al `ToggleButton` el aspecto de "botón cuadrado gris" de Windows y dejar solo el texto de la estrella.
- **Triggers:** El cambio de color de Gris a Dorado ocurre automáticamente en el XAML gracias al `Trigger` que debe vigilar una propiedad que almacena el estado (`IsChecked`) del botón (estrella).

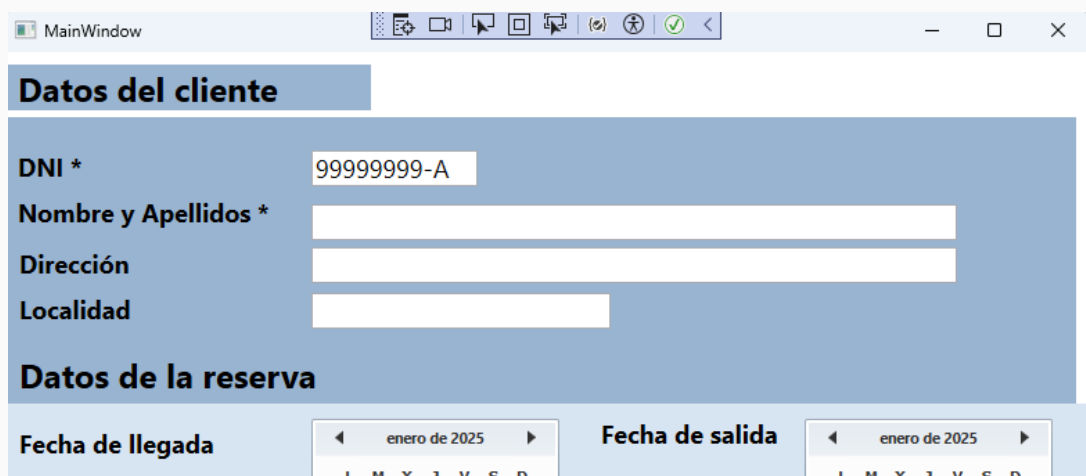
Paso 4. Usar el Control de usuario

Vamos a hacerlo en nuestro formulario de reservas. Para ello debemos insertar el control mediante el siguiente código en el archivo xaml:



Q Sandra Alonso Díaz (Elaboración propia)

Nos aparecerá el control y podremos moverlo donde nos interese y cambiarle el tamaño. Comprobamos su funcionamiento al ejecutar el proyecto:



303112345
6789101112
13141516171819
20212223242526
272829303112
3456789

Nº de habitaciones

Tipo de habitaciones

Régimen de alojamiento

☐ Alojamiento y desayuno

☐ Media Pensión

☒ Pensión Completa

Cancelar

Aceptar

303112345
6789101112
13141516171819
20212223242526
272829303112
3456789

☒ Fumador

En virtud de la ley de sanidad se informa a los clientes de que solo podrán fumar en habitaciones habilitadas a tal fin

★

★

★

★

★

Q Sandra Alonso Díaz (Elaboración propia)

Así, habremos finalizado el proceso de creación de nuestro primer control personalizado.

